Aula 4 — EDA Profissional (Versão Avançada) - Para Aula

Dataset: Airbnb NYC

Objetivos Desta Aula

- **Explorar um dataset real de mercado:** Entender a estrutura e o conteúdo de dados reais.
- **Detectar e tratar valores faltantes e outliers:** Aprender técnicas para limpar e preparar os dados.
- **Realizar análise por categoria e por localização:** Investigar padrões e tendências em diferentes segmentos e regiões.
- **Construir gráficos profissionais e informativos:** Utilizar bibliotecas avançadas para visualizações de alta qualidade.
- **Introduzir correlação entre variáveis:** Entender as relações entre diferentes atributos do dataset.
- **Engenharia de Features:** Criar novas variáveis para enriquecer a análise.
- **Análise Geoespacial:** Visualizar dados em um contexto geográfico.
- **Interpretar e comunicar insights:** Extrair conclusões significativas dos dados.

1) Importação das bibliotecas Essenciais

Nesta seção, importamos todas as bibliotecas Python necessárias para a Análise Exploratória de Dados (EDA).

- `pandas` : Para manipulação e análise de dados em formato de DataFrame.
- `matplotlib.pyplot` : Para a criação de gráficos estáticos.
- `seaborn` : Baseado no matplotlib, oferece uma interface de alto nível para criar gráficos estatísticos atraentes e informativos.
- `numpy` : Para operações numéricas e matemáticas, especialmente útil para lidar com arrays e cálculos estatísticos.
- `folium` : Para criar mapas interativos e visualizar dados geoespaciais.

In [1]:

```
# Importe aqui as bibliotecas necessárias: pandas, matplotlib.pyplot, seaborn,
numpy, folium
# Configure os estilos de visualização (sns.set_style, plt.rcParams)

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import folium

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
```

2) Carregar o Dataset

Carregamos o dataset do Airbnb NYC a partir de um arquivo CSV. Este é o primeiro passo para qualquer análise de dados, onde os dados brutos são lidos e transformados em um DataFrame do pandas para fácil manipulação.

In [2]:

```
# Carregue o arquivo CSV "AB_NYC_2019.csv" em um DataFrame do pandas
# Exiba as primeiras linhas do DataFrame para uma visão inicial dos dados

df = pd.read_csv("AB_NYC_2019.csv")
display(df.head())
```

	id	name	host_id	host_name	neighbourhood_group	neighbo
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensing
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtow
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Ha

3) Visão Geral e Estrutura dos Dados

Esta seção fornece uma visão geral rápida do dataset, incluindo suas dimensões, tipos de dados e um resumo estatístico. Isso nos ajuda a entender a estrutura dos dados e identificar possíveis problemas, como valores ausentes ou tipos de dados incorretos.

In [3]:

```
# Exiba o número de linhas e colunas do DataFrame (df.shape)
# Exiba informações sobre o DataFrame, incluindo tipos de dados e valores não-
nulos (df.info())
# Exiba estatísticas descritivas para as colunas numéricas (df.describe())
# Exiba estatísticas descritivas para todas as colunas (incluindo categóricas)
(df.describe(include="all"))

display(df.shape)
display(df.info())
display(df.describe())
display(df.describe(include="all"))
```

```
(48895, 16)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              48895 non-null  int64
 1   name                            48879 non-null  object
 2   host_id                         48895 non-null  int64
 3   host_name                       48874 non-null  object
 4   neighbourhood_group             48895 non-null  object
 5   neighbourhood                   48895 non-null  object
 6   latitude                        48895 non-null  float64
 7   longitude                       48895 non-null  float64
 8   room_type                       48895 non-null  object
 9   price                           48895 non-null  int64
 10  minimum_nights                  48895 non-null  int64
 11  number_of_reviews               48895 non-null  int64
 12  last_review                     38843 non-null  object
 13  reviews_per_month               38843 non-null  float64
 14  calculated_host_listings_count  48895 non-null  int64
 15  availability_365                48895 non-null  int64
dtypes: float64(3), int64(7), object(6)
memory usage: 6.0+ MB

None
```

	id	host_id	latitude	longitude	price
count	4.889500e+04	4.889500e+04	48895.000000	48895.000000	48895.000000
mean	1.901714e+07	6.762001e+07	40.728949	-73.952170	152.720687
std	1.098311e+07	7.861097e+07	0.054530	0.046157	240.154170
min	2.539000e+03	2.438000e+03	40.499790	-74.244420	0.000000
25%	9.471945e+06	7.822033e+06	40.690100	-73.983070	69.000000
50%	1.967728e+07	3.079382e+07	40.723070	-73.955680	106.000000
75%	2.915218e+07	1.074344e+08	40.763115	-73.936275	175.000000
max	3.648724e+07	2.743213e+08	40.913060	-73.712990	10000.000000

	id	name	host_id	host_name	neighbourhood_grou
count	4.889500e+04	48879	4.889500e+04	48874	48895
unique	NaN	47905	NaN	11452	5
top	NaN	Hillside Hotel	NaN	Michael	Manhattan
freq	NaN	18	NaN	417	21661
mean	1.901714e+07	NaN	6.762001e+07	NaN	NaN
std	1.098311e+07	NaN	7.861097e+07	NaN	NaN
min	2.539000e+03	NaN	2.438000e+03	NaN	NaN
25%	9.471945e+06	NaN	7.822033e+06	NaN	NaN
50%	1.967728e+07	NaN	3.079382e+07	NaN	NaN
75%	2.915218e+07	NaN	1.074344e+08	NaN	NaN
max	3.648724e+07	NaN	2.743213e+08	NaN	NaN

4) Tratamento de Valores Faltantes

Valores faltantes são comuns em datasets reais e podem afetar a qualidade da análise. Nesta seção, identificamos e tratamos esses valores, decidindo entre remover, preencher com um valor específico (média, mediana, moda) ou usar métodos mais avançados.

In [4]:

```
# Calcule e exiba a quantidade de valores faltantes por coluna
print("Valores faltantes antes do tratamento:\n", df.isnull().sum())

# Preencha os valores faltantes nas colunas `name` e `host_name` com "unknown"
df['name'] = df['name'].fillna('unknown')
df['host_name'] = df['host_name'].fillna('unknown')

# Preencha os valores faltantes na coluna `reviews_per_month` com 0
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)

# Converta a coluna `last_review` para datetime e preencha os valores faltantes
com uma data de referência (e.g., "2010-01-01")
df['last_review'] = pd.to_datetime(df['last_review'], errors='coerce')
df['last_review'] = df['last_review'].fillna(pd.to_datetime('2010-01-01'))

# Verifique novamente os valores faltantes após o tratamento
print("\nValores faltantes após o tratamento:\n", df.isnull().sum())
```

```
Valores faltantes antes do tratamento:
 id                                     0
name                                  16
host_id                                0
host_name                             21
neighbourhood_group                    0
neighbourhood                          0
latitude                               0
longitude                              0
room_type                              0
price                                  0
minimum_nights                         0
number_of_reviews                      0
last_review                        10052
reviews_per_month                  10052
calculated_host_listings_count         0
availability_365                       0
dtype: int64

Valores faltantes após o tratamento:
 id                                 0
name                               0
host_id                            0
host_name                          0
neighbourhood_group                0
neighbourhood                      0
latitude                           0
longitude                          0
room_type                          0
price                              0
minimum_nights                     0
number_of_reviews                  0
last_review                        0
reviews_per_month                  0
calculated_host_listings_count     0
availability_365                   0
dtype: int64
```

5) Detecção e Tratamento de Outliers

Outliers são pontos de dados que se desviam significativamente de outras observações. Eles podem distorcer análises estatísticas e modelos de machine learning. Nesta seção, identificamos e tratamos outliers em variáveis numéricas chave, como `price` e `minimum_nights`. Usaremos o método do Intervalo Interquartil (IQR) para uma abordagem robusta.

In [5]:

```
# Defina uma função `remove_outliers_iqr` para detectar e remover outliers usando
o método IQR
# Aplique a função para tratar outliers nas colunas `price`, `minimum_nights` e
`number_of_reviews`
# Exiba as estatísticas descritivas antes e depois do tratamento para cada coluna
# Atualize o DataFrame principal com os dados limpos

def remove_outliers_iqr(df, column):
    print(f"\nEstatísticas descritivas para '{column}' antes do tratamento de
outliers:")
    display(df[column].describe())

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_filtered = df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]

    print(f"\nEstatísticas descritivas para '{column}' depois do tratamento de
outliers:")
    display(df_filtered[column].describe())

    return df_filtered

# Aplicar a função para tratar outliers
df_cleaned = df.copy()
df_cleaned = remove_outliers_iqr(df_cleaned, 'price')
df_cleaned = remove_outliers_iqr(df_cleaned, 'minimum_nights')
df_cleaned = remove_outliers_iqr(df_cleaned, 'number_of_reviews')

df = df_cleaned.copy() # Atualiza o DataFrame principal com os dados limpos
```

```
Estatísticas descritivas para 'price' antes do tratamento de outliers:
```

	price
count	48895.000000
mean	152.720687
std	240.154170
min	0.000000
25%	69.000000
50%	106.000000
75%	175.000000
max	10000.000000

dtype: float64

```
Estatísticas descritivas para 'price' depois do tratamento de outliers:
```

	price
count	45923.000000
mean	119.970320
std	68.150148
min	0.000000
25%	65.000000
50%	100.000000
75%	159.000000
max	334.000000

dtype: float64

```
Estatísticas descritivas para 'minimum_nights' antes do tratamento de
outliers:
```

	minimum_nights
count	45923.000000
mean	6.938070
std	19.857823
min	1.000000
25%	1.000000
50%	2.000000
75%	5.000000
max	1250.000000

dtype: float64

```
Estatísticas descritivas para 'minimum_nights' depois do tratamento de
outliers:
```

	minimum_nights
count	39738.000000
mean	2.696160
std	1.863537
min	1.000000
25%	1.000000
50%	2.000000

	minimum_nights
75%	3.000000
max	11.000000

dtype: float64

```
Estatísticas descritivas para 'number_of_reviews' antes do tratamento de
outliers:
```

	number_of_reviews
count	39738.000000
mean	26.377548
std	47.474653
min	0.000000
25%	1.000000
50%	7.000000
75%	29.000000
max	629.000000

dtype: float64

```
Estatísticas descritivas para 'number_of_reviews' depois do tratamento de
outliers:
```

	number_of_reviews
count	35202.000000
mean	12.362536
std	16.631476
min	0.000000
25%	1.000000
50%	5.000000
75%	17.000000
max	71.000000

dtype: float64

6) Análise Exploratória de Dados (EDA) Aprofundada

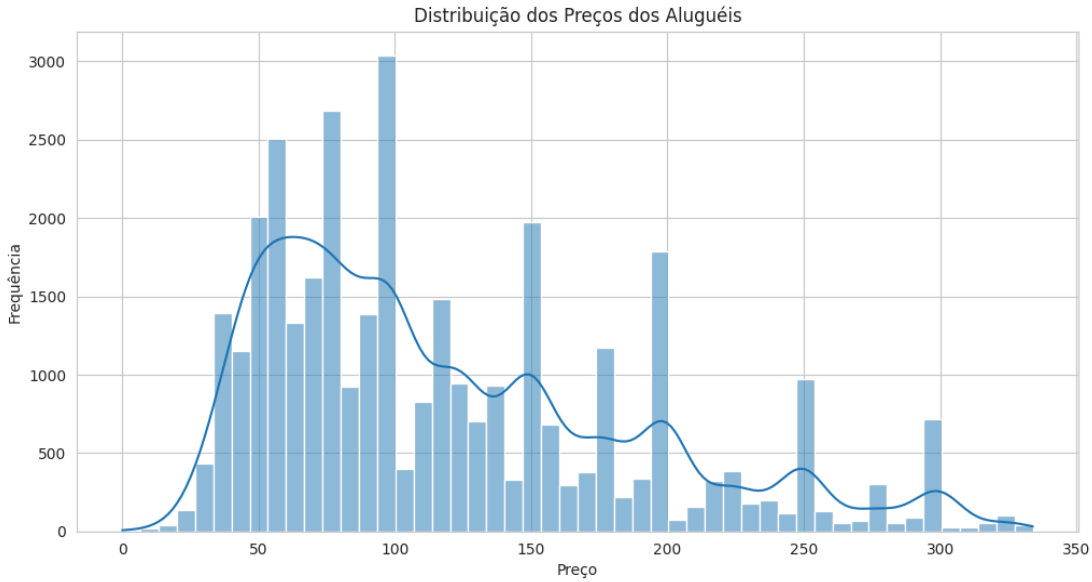
Com os dados limpos, podemos agora aprofundar nossa análise, explorando as distribuições das variáveis, as relações entre elas e identificando padrões que podem gerar insights valiosos.

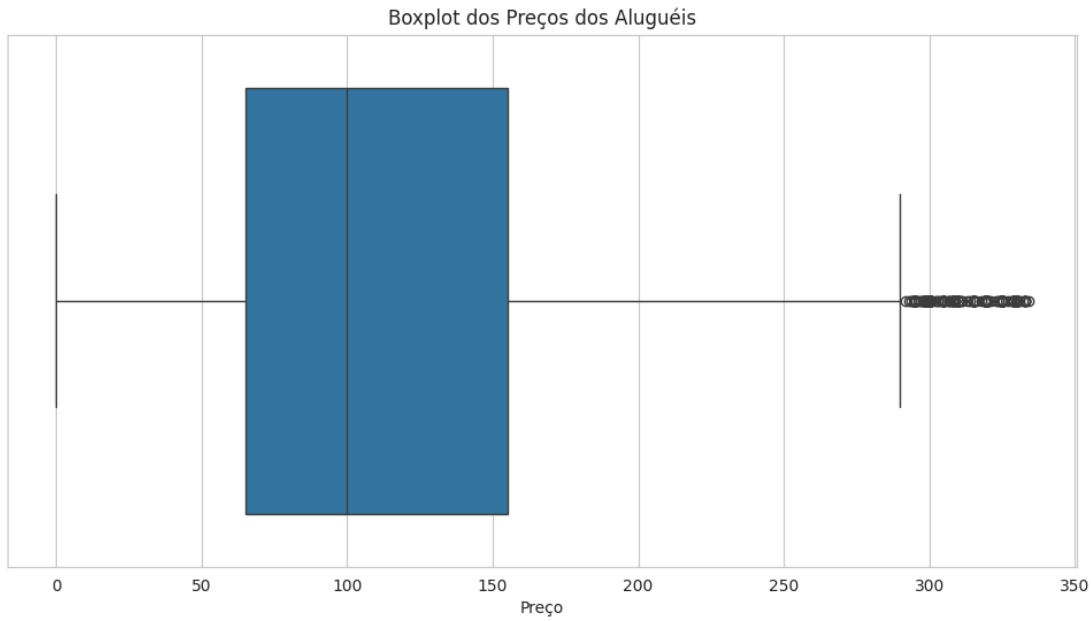
6.1) Distribuição de Preços

Vamos visualizar a distribuição dos preços dos aluguéis para entender a faixa de valores e identificar a concentração de preços.

In [7]:

```
# Crie um histograma da coluna `price` usando `sns.histplot`
# Crie um boxplot da coluna `price` usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos

# Histograma da coluna price
plt.figure(figsize=(12, 6))
sns.histplot(df['price'], bins=50, kde=True)
plt.title('Distribuição dos Preços dos Aluguéis')
plt.xlabel('Preço')
plt.ylabel('Frequência')
plt.show()

# Boxplot da coluna price
plt.figure(figsize=(12, 6))
sns.boxplot(x=df['price'])
plt.title('Boxplot dos Preços dos Aluguéis')
plt.xlabel('Preço')
plt.show()
```





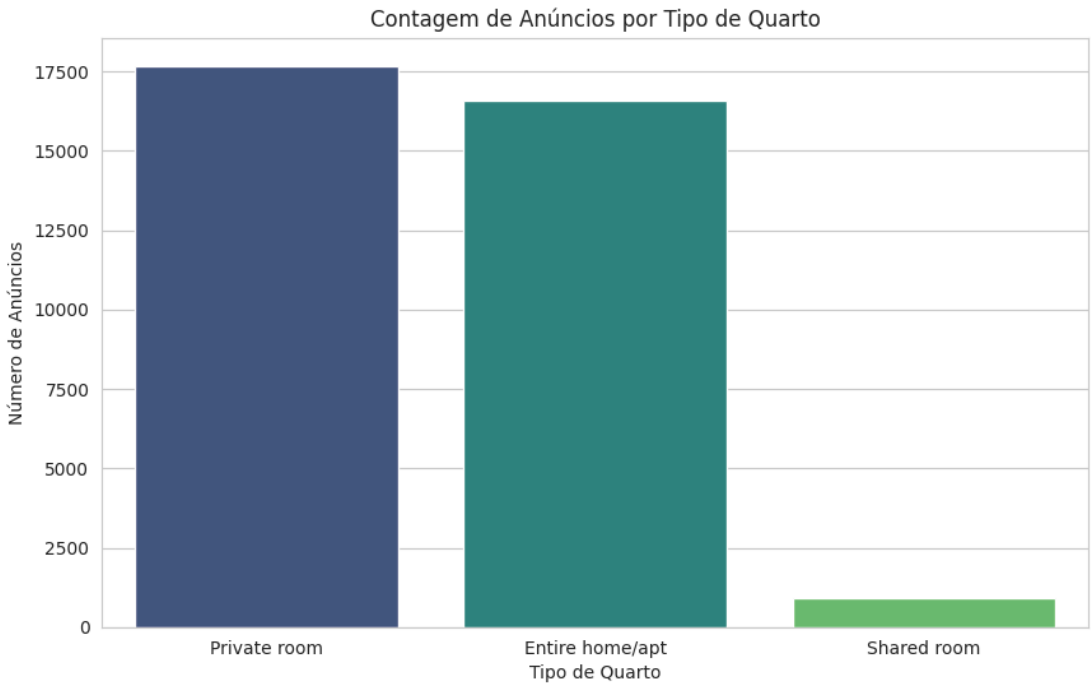
6.2) Análise por Tipo de Quarto (`room_type`)

Entender a distribuição dos tipos de quartos e como eles se relacionam com o preço é crucial para proprietários e hóspedes.

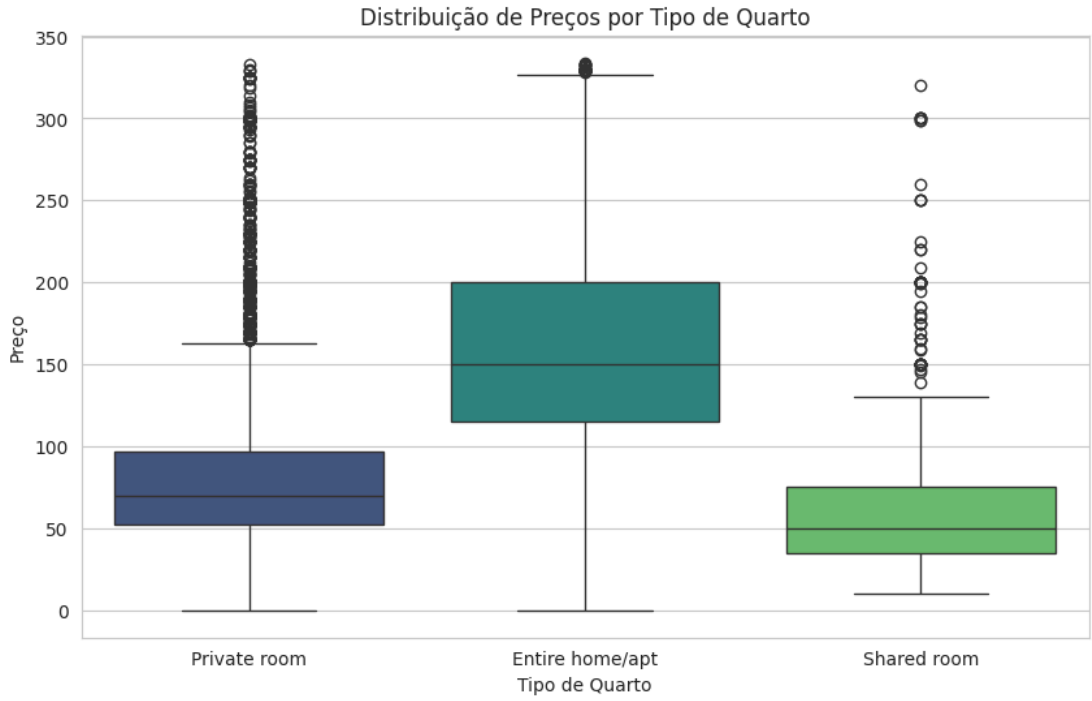
In [8]:

```
# Crie um gráfico de contagem dos tipos de quarto usando `sns.countplot`
# Crie um boxplot do preço por tipo de quarto usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos

# Gráfico de contagem dos tipos de quarto
plt.figure(figsize=(10, 6))
sns.countplot(x='room_type', data=df, palette='viridis')
plt.title('Contagem de Anúncios por Tipo de Quarto')
plt.xlabel('Tipo de Quarto')
plt.ylabel('Número de Anúncios')
plt.show()

# Boxplot do preço por tipo de quarto
plt.figure(figsize=(10, 6))
sns.boxplot(x='room_type', y='price', data=df, palette='viridis')
plt.title('Distribuição de Preços por Tipo de Quarto')
plt.xlabel('Tipo de Quarto')
plt.ylabel('Preço')
plt.show()
```

```
/tmp/ipykernel_566/2697205568.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(x='room_type', data=df, palette='viridis')
```




```
/tmp/ipykernel_566/2697205568.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.boxplot(x='room_type', y='price', data=df, palette='viridis')
```



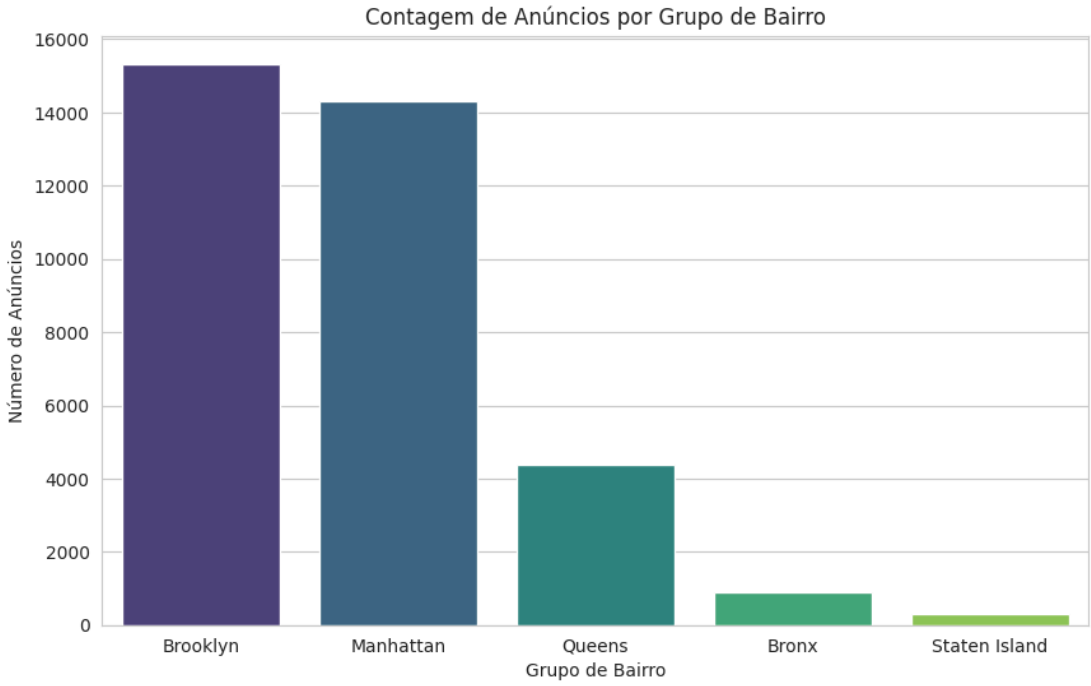
6.3) Análise por Grupo de Bairro (`neighbourhood_group`)

A localização é um dos fatores mais importantes no mercado imobiliário. Vamos analisar como os preços e a disponibilidade variam entre os diferentes grupos de bairros de NYC.

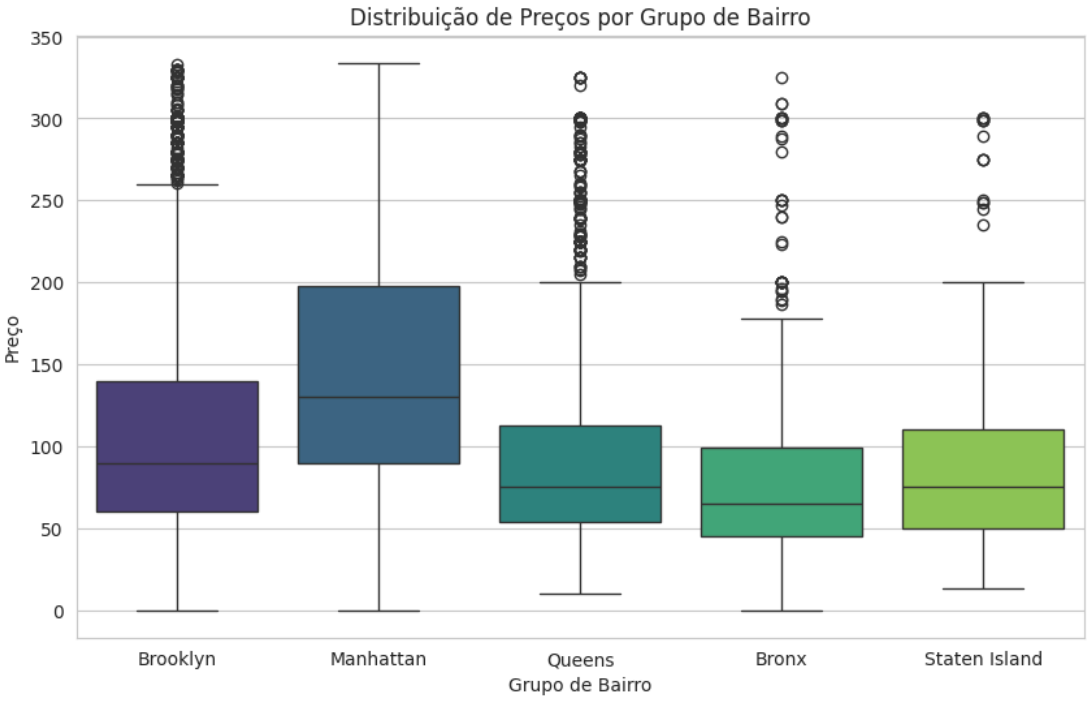
In [9]:

```
# Crie um gráfico de contagem de anúncios por grupo de bairro usando
`sns.countplot`
# Crie um boxplot do preço por grupo de bairro usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos

# Gráfico de contagem de anúncios por grupo de bairro
plt.figure(figsize=(10, 6))
sns.countplot(x='neighbourhood_group', data=df, palette='viridis')
plt.title('Contagem de Anúncios por Grupo de Bairro')
plt.xlabel('Grupo de Bairro')
plt.ylabel('Número de Anúncios')
plt.show()

# Boxplot do preço por grupo de bairro
plt.figure(figsize=(10, 6))
sns.boxplot(x='neighbourhood_group', y='price', data=df, palette='viridis')
plt.title('Distribuição de Preços por Grupo de Bairro')
plt.xlabel('Grupo de Bairro')
plt.ylabel('Preço')
plt.show()
```

```
/tmp/ipykernel_566/2944683959.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.countplot(x='neighbourhood_group', data=df, palette='viridis')
```




```
/tmp/ipykernel_566/2944683959.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.boxplot(x='neighbourhood_group', y='price', data=df,
palette='viridis')
```



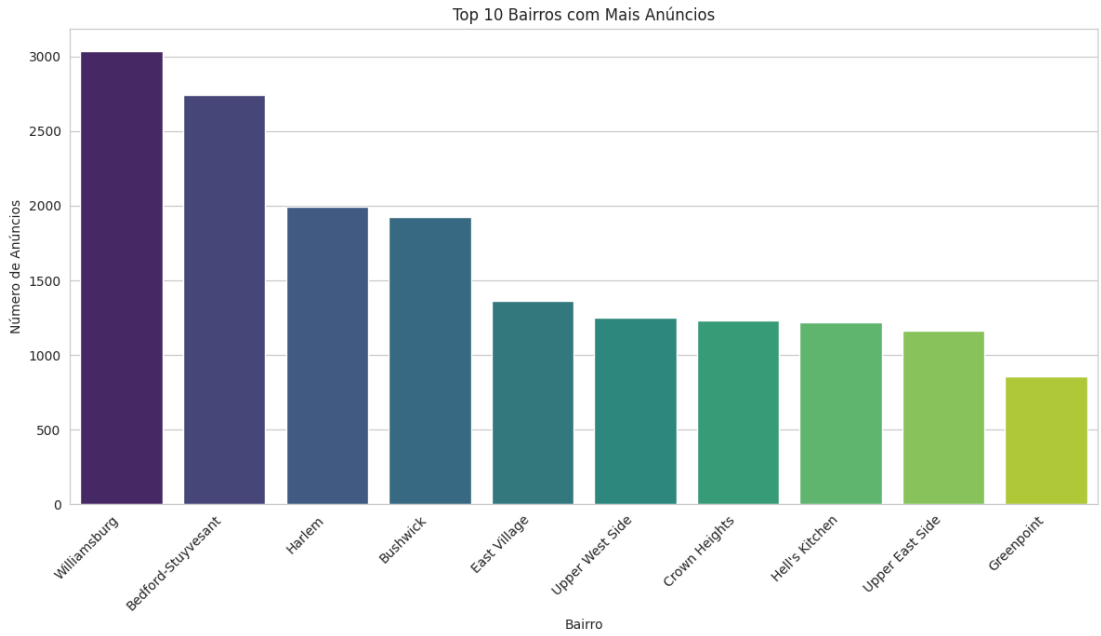
6.4) Top 10 Bairros com Mais Anúncios

Identificar os bairros com maior concentração de anúncios pode indicar áreas de alta demanda ou popularidade.

In [10]:

```
# Calcule os 10 bairros com mais anúncios
# Crie um gráfico de barras para visualizar esses bairros usando `sns.barplot`
# Adicione títulos e rótulos aos gráficos, e rotacione os rótulos do eixo X se
necessário

# 10 bairros com mais anúncios
top_10_neighbourhoods = df['neighbourhood'].value_counts().head(10)

# Crie um gráfico de barras para visualizar esses bairros
plt.figure(figsize=(12, 7))
sns.barplot(x=top_10_neighbourhoods.index, y=top_10_neighbourhoods.values,
palette='viridis')
plt.title('Top 10 Bairros com Mais Anúncios')
plt.xlabel('Bairro')
plt.ylabel('Número de Anúncios')
plt.xticks(rotation=45, ha='right') # Rotaciona os rótulos do eixo X para melhor
legibilidade
plt.tight_layout() # Ajusta o layout para evitar sobreposição
plt.show()
```

```
/tmp/ipykernel_566/2902852416.py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(x=top_10_neighbourhoods.index,
y=top_10_neighbourhoods.values, palette='viridis')
```



6.5) Análise Geoespacial dos Anúncios

Visualizar os anúncios em um mapa nos permite identificar clusters geográficos e entender a distribuição espacial dos imóveis.

In [13]:

```
from folium.plugins import MarkerCluster

# Coordenadas do centro de Nova York para o mapa base
nyc_coords = [40.730610, -73.935242]

# Crie um mapa base de Nova York
m = folium.Map(location=nyc_coords, zoom_start=11)

# Adicione um MarkerCluster ao mapa para agrupar os marcadores
marker_cluster = MarkerCluster().add_to(m)

# Amostre uma parte do DataFrame (e.g., 5000 pontos) para melhor performance na
visualização
df_sample = df.sample(n=5000, random_state=42) if len(df) > 5000 else df

# Adicione marcadores ao mapa para cada ponto amostrado
for idx, row in df_sample.iterrows():
    # Cria o texto do popup com informações relevantes
    popup_text = f"""
    <b>Name:</b> {row['name']}<br>
    <b>Host Name:</b> {row['host_name']}<br>
    <b>Neighbourhood Group:</b> {row['neighbourhood_group']}<br>
    <b>Neighbourhood:</b> {row['neighbourhood']}<br>
    <b>Room Type:</b> {row['room_type']}<br>
    <b>Price:</b> ${row['price']}<br>
    <b>Minimum Nights:</b> {row['minimum_nights']}<br>
    <b>Number of Reviews:</b> {row['number_of_reviews']}
    """
    folium.Marker(
        location=[row['latitude'], row['longitude']],
        popup=popup_text,
        tooltip=row['name'] # Tooltip para mostrar o nome ao passar o mouse
    ).add_to(marker_cluster)

# Salve o mapa em um arquivo HTML
map_file = "airbnb_nyc_map.html"
m.save(map_file)
print(f"Mapa salvo em: {map_file}")

# (Opcional) Exiba o mapa diretamente no notebook se o ambiente suportar
display(m)
```

Output hidden; open in https://colab.research.google.com to view.

7) Engenharia de Features

A engenharia de features envolve a criação de novas variáveis a partir das existentes para melhorar a capacidade preditiva dos modelos ou para obter novos insights. Aqui, criaremos algumas features simples.

7.1) `host_experience_days`

Calcularemos a experiência do anfitrião em dias, baseando-nos na data da última avaliação (`last_review`) e uma data de referência (ou a data da primeira avaliação, se disponível). Isso pode indicar a longevidade e experiência do anfitrião na plataforma.

In [14]:
```
# Calcule a data mais recente de `last_review`
most_recent_review_date = df['last_review'].max()

# Crie a nova feature `host_experience_days` calculando a diferença em dias entre
a data mais recente e `last_review`
df['host_experience_days'] = (most_recent_review_date - df['last_review']).dt.days

# Exiba as estatísticas descritivas da nova feature
print("Estatísticas descritivas para 'host_experience_days':")
display(df['host_experience_days'].describe())
```

```
Estatísticas descritivas para 'host_experience_days':
```

	host_experience_days
count	35202.000000
mean	912.940032
std	1301.936154
min	0.000000
25%	22.000000
50%	190.000000
75%	1150.000000
max	3475.000000

dtype: float64

7.2) `price_per_night_per_review`

Esta feature pode indicar o custo-benefício percebido, relacionando o preço por noite com o número de avaliações. Um valor baixo pode indicar um bom negócio, enquanto um valor alto pode sugerir um preço elevado para poucos reviews.

In [15]:

```
# Crie a nova feature `price_per_night_per_review`, dividindo `price` por
`number_of_reviews`
# Lembre-se de tratar a divisão por zero (e.g., se `number_of_reviews` for 0, use
o próprio `price`)
df['price_per_night_per_review'] = np.where(
    df['number_of_reviews'] == 0,
    df['price'],
    df['price'] / df['number_of_reviews']
)

# Exiba as estatísticas descritivas da nova feature
print("Estatísticas descritivas para 'price_per_night_per_review':")
display(df['price_per_night_per_review'].describe())
```

```
Estatísticas descritivas para 'price_per_night_per_review':
```

	price_per_night_per_review
count	35202.000000
mean	49.967902
std	63.791240
min	0.000000
25%	5.801613
50%	21.966667
75%	70.000000
max	333.000000

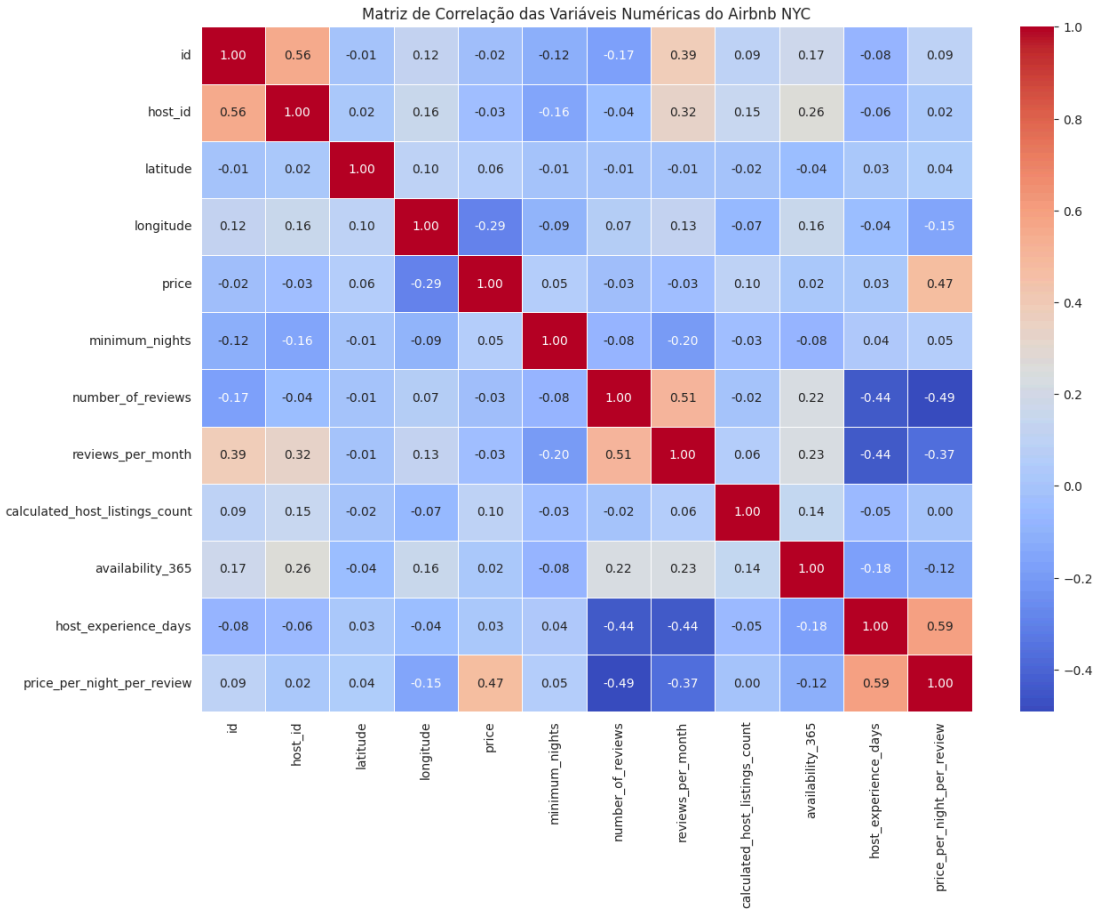
dtype: float64

8) Análise de Correlação

A correlação mede a força e a direção da relação linear entre duas variáveis numéricas. Um mapa de calor (heatmap) da matriz de correlação é uma excelente ferramenta para visualizar essas relações.

In [16]:

```
# Selecione apenas as colunas numéricas do DataFrame
numeric_df = df.select_dtypes(include=[np.number])

# Calcule a matriz de correlação
correlation_matrix = numeric_df.corr()

# Crie um heatmap da matriz de correlação usando `sns.heatmap`
plt.figure(figsize=(14, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f",
linewidths=.5)
plt.title('Matriz de Correlação das Variáveis Numéricas do Airbnb NYC')
plt.show()
```



Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.